

Guia de criação dos usuários de serviço do homelab JMSalles

1. Objetivo

Este documento descreve como criar os usuários de serviço necessários para a esteira do homelab JMSalles.

Usuários contemplados:

```
svc-gitea-runner  
svc-k8s-pull  
svc-docker-deploy
```

Uso de cada usuário:

Usuário	Local	Finalidade
svc-gitea-runner	Nexus	Usado pela esteira do Gitea/act_runner para fazer login e push de imagens no registry
svc-k8s-pull	Nexus	Usado pelo Kubernetes para fazer pull de imagens privadas
svc-docker-deploy	Linux no docker-hml	Usado pela esteira para executar deploy em Docker/HML

Neste modelo, o usuário **svc-docker-deploy** terá:

```
Login via chave SSH  
Login via usuário e senha  
Permissão direta ao Docker via grupo docker
```

Observação importante:

```
Usuário no grupo docker possui privilégio elevado no host.  
Na prática, acesso ao grupo docker pode equivaler a acesso administrativo.  
Para o homelab e esteira controlada, seguiremos com essa abordagem simplificada.
```

2. Visão geral do fluxo

```
Gitea  
|  
act_runner
```

```
|  
Build da imagem  
|  
Login no registry usando svc-gitea-runner  
|  
Push para registry.jmsalles.homelab.br  
|  
+--> Deploy Docker HML usando svc-docker-deploy  
|  
+--> Deploy Kubernetes HML usando svc-k8s-pull
```

Registry usado no projeto:

```
registry.jmsalles.homelab.br
```

Interface web do Nexus:

```
https://nexus.jmsalles.homelab.br
```

Servidor Docker HML:

```
docker-hml.jmsalles.homelab.br
```

IP:

```
192.168.31.37
```

3. Usuários que serão criados

3.1 `svc-gitea-runner`

Finalidade:

```
Usado pela pipeline do Gitea Actions/act_runner para fazer push de imagens  
Docker no Nexus.
```

Permissões necessárias no Nexus:

```
browse  
read
```

```
add  
edit
```

Repositório principal:

```
docker-hosted
```

Endpoint usado:

```
registry.jmsalles.homelab.br
```

Exemplo de uso futuro:

```
docker login registry.jmsalles.homelab.br -u svc-gitea-runner
```

```
docker push registry.jmsalles.homelab.br/lab/app-teste:latest
```

3.2 `svc-k8s-pull`

Finalidade:

```
Usado pelo Kubernetes para baixar imagens privadas do Nexus.
```

Permissões necessárias no Nexus:

```
browse  
read
```

Repositório principal:

```
docker-hosted
```

Exemplo de uso futuro no Kubernetes:

```
kubectl create secret docker-registry nexus-registry \
  --docker-server=registry.jmsalles.homelab.br \
  --docker-username=svc-k8s-pull \
  --docker-password='<SENHA_DO_USUARIO>' \
  --docker-email='jefersonmattossalles@gmail.com' \
  -n <namespace>
```

3.3 svc-docker-deploy

Finalidade:

Usado pela esteira para realizar deploy controlado no ambiente Docker HML.

Esse usuário será criado no Linux do servidor:

docker-hml.jmsalles.homelab.br

Ele será usado para executar ações como:

```
docker ps
docker compose ps
docker compose pull
docker compose up -d
docker compose restart
docker compose logs
```

Neste modelo, ele será adicionado ao grupo:

docker

Também será permitido login por:

Chave SSH
Usuário e senha

Parte 1: Criar roles no Nexus

4. Acessar o Nexus

Acesse:

```
https://nexus.jmsalles.homelab.br
```

Entre com usuário administrativo.

Normalmente:

```
admin
```

5. Criar role para push no Docker hosted

No Nexus, acesse:

```
Settings
Security
Roles
Create role
Nexus role
```

Preencha:

Campo	Valor
Role ID	role-docker-hosted-push
Role name	Role Docker Hosted Push
Description	Permissao para push e pull no repositorio docker-hosted

Em **Privileges**, procure por:

```
nx-repository-view-docker-docker-hosted
```

Adicione:

```
nx-repository-view-docker-docker-hosted-browse
nx-repository-view-docker-docker-hosted-read
nx-repository-view-docker-docker-hosted-add
nx-repository-view-docker-docker-hosted-edit
```

Não adicione **delete** inicialmente.

Motivo:

O usuario da esteira deve conseguir publicar e atualizar imagens, mas nao precisa apagar artefatos.

Salve a role.

6. Criar role para pull no Docker hosted

No Nexus, acesse:

Settings
Security
Roles
Create role
Nexus role

Preencha:

Campo	Valor
Role ID	role-docker-hosted-pull
Role name	Role Docker Hosted Pull
Description	Permissao somente leitura no repositorio docker-hosted

Em **Privileges**, adicione:

nx-repository-view-docker-docker-hosted-browse
nx-repository-view-docker-docker-hosted-read

Salve a role.

Essa role será usada pelo usuário:

svc-k8s-pull

Parte 2: Criar usuários no Nexus

7. Criar usuário `svc-gitea-runner`

No Nexus, acesse:

```
Settings
Security
Users
Create local user
```

Preencha:

Campo	Valor
ID	<code>svc-gitea-runner</code>
First name	<code>svc</code>
Last name	<code>gitea-runner</code>
Email	<code>jefersonmattossalles@gmail.com</code>
Status	<code>Active</code>
Password	<code><DEFINIR_SENHA_FORTE></code>
Confirm password	<code><REPETIR_SENHA></code>

Em `Roles`, adicione:

```
role-docker-hosted-push
```

Salve.

Uso esperado:

```
Esse usuario sera usado pelo act_runner para fazer docker login e docker push no registry.
```

8. Criar usuário `svc-k8s-pull`

No Nexus, acesse:

```
Settings
Security
Users
Create local user
```

Preencha:

Campo	Valor
ID	svc-k8s-pull
First name	svc
Last name	k8s-pull
Email	jefersonmattossalles@gmail.com
Status	Active
Password	<DEFINIR_SENHA_FORTE>
Confirm password	<REPETIR_SENHA>

Em **Roles**, adicione:

```
role-docker-hosted-pull
```

Salve.

Uso esperado:

```
Esse usuario sera usado pelo Kubernetes para baixar imagens privadas do
registry.
```

9. Validar Docker Bearer Token Realm

Para o Docker login funcionar corretamente, valide o realm do Docker.

No Nexus, acesse:

```
Settings
Security
Realms
```

Em **Available**, localize:


```
Docker Bearer Token Realm
```

Mova para **Active**.

Salve.

Parte 3: Testar os usuários do Nexus

10. Testar login com **svc-gitea-runner**

No servidor ou cliente Docker configurado com o registry, execute:

```
docker login registry.jmsalles.homelab.br -u svc-gitea-runner
```

Informe a senha criada no Nexus.

Resultado esperado:

```
Login Succeeded
```

11. Testar push com **svc-gitea-runner**

Baixe uma imagem pequena:

```
docker pull alpine:latest
```

Crie a tag:

```
docker tag alpine:latest registry.jmsalles.homelab.br/lab/alpine:svc-gitea-runner-test
```

Faça push:

```
docker push registry.jmsalles.homelab.br/lab/alpine:svc-gitea-runner-test
```

Resultado esperado:

```
Imagem enviada com sucesso para o Nexus.
```

Valide no Nexus:

```
Browse  
docker-hosted  
lab/alpine
```

12. Testar pull com `svc-k8s-pull`

Faça logout do usuário anterior:

```
docker logout registry.jmsalles.homelab.br
```

Faça login com o usuário de leitura:

```
docker login registry.jmsalles.homelab.br -u svc-k8s-pull
```

Faça pull da imagem publicada:

```
docker pull registry.jmsalles.homelab.br/lab/alpine:svc-gitea-runner-test
```

Resultado esperado:

```
Imagem baixada com sucesso.
```

13. Validar que `svc-k8s-pull` não consegue fazer push

Crie uma nova tag:

```
docker tag alpine:latest registry.jmsalles.homelab.br/lab/alpine:svc-k8s-pull-  
test
```

Tente fazer push com o usuário `svc-k8s-pull`:

```
docker push registry.jmsalles.homelab.br/lab/alpine:svc-k8s-pull-test
```

Resultado esperado:

```
denied
unauthorized
insufficient permissions
```

Esse erro é esperado.

Motivo:

```
O usuario svc-k8s-pull deve ter somente permissao de leitura.
```

Parte 4: Criar usuário Linux `svc-docker-deploy`

14. Criar usuário no servidor Docker HML

Acesse o servidor:

```
ssh jmsalles@docker-hml.jmsalles.homelab.br
```

Crie o usuário:

```
sudo useradd -m -s /bin/bash svc-docker-deploy
```

Defina senha para permitir login com usuário e senha:

```
sudo passwd svc-docker-deploy
```

Informe uma senha forte.

Valide o usuário:

```
id svc-docker-deploy
```

Valide o shell:

```
getent passwd svc-docker-deploy
```

Resultado esperado:

```
svc-docker-deploy:x:<UID>:<GID>::/home/svc-docker-deploy:/bin/bash
```

15. Criar chave pública e privada para **svc-docker-deploy**

A chave deve ser criada na máquina que vai executar a esteira, por exemplo:

```
Servidor do Gitea act_runner  
Notebook administrativo  
Servidor de CI/CD
```

A chave privada ficará na esteira ou futuramente no Vault.

A chave pública será adicionada no servidor Docker HML, no usuário:

```
svc-docker-deploy
```

15.1 Criar diretório local para as chaves

Na máquina de origem, execute:

```
mkdir -p ~/.ssh/jmsalles-homelab
```

```
chmod 700 ~/.ssh
```

```
chmod 700 ~/.ssh/jmsalles-homelab
```

15.2 Gerar a chave SSH

Crie a chave:

```
ssh-keygen -t ed25519 -f ~/.ssh/jmsalles-homelab/svc-docker-deploy -C "svc-docker-deploy@docker-hml"
```

Quando pedir senha da chave, para uso automatizado pela esteira pode deixar sem senha.

Arquivos criados:

```
~/.ssh/jmsalles-homelab/svc-docker-deploy  
~/.ssh/jmsalles-homelab/svc-docker-deploy.pub
```

Ajuste as permissões:

```
chmod 600 ~/.ssh/jmsalles-homelab/svc-docker-deploy
```

```
chmod 644 ~/.ssh/jmsalles-homelab/svc-docker-deploy.pub
```

Valide:

```
ls -l ~/.ssh/jmsalles-homelab/
```

15.3 Exibir a chave pública

Exiba a chave pública:

```
cat ~/.ssh/jmsalles-homelab/svc-docker-deploy.pub
```

Copie o conteúdo completo da chave pública.

O formato será parecido com:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExemploDeChavePublica svc-docker-deploy@docker-hml
```

15.4 Criar o diretório `.ssh` no usuário `svc-docker-deploy`

No servidor Docker HML, execute:

```
sudo mkdir -p /home/svc-docker-deploy/.ssh
```

Crie ou edite o arquivo `authorized_keys`:

```
sudo vim /home/svc-docker-deploy/.ssh/authorized_keys
```

Cole a chave pública gerada anteriormente.

Ajuste o dono:

```
sudo chown -R svc-docker-deploy:svc-docker-deploy /home/svc-docker-deploy/.ssh
```

Ajuste as permissões:

```
sudo chmod 700 /home/svc-docker-deploy/.ssh
```

```
sudo chmod 600 /home/svc-docker-deploy/.ssh/authorized_keys
```

Valide:

```
sudo ls -lah /home/svc-docker-deploy/.ssh
```

15.5 Testar acesso SSH com chave

Na máquina onde a chave privada foi criada, teste:

```
ssh -i ~/.ssh/jmsalles-homelab/svc-docker-deploy svc-docker-deploy@docker-hml.jmsalles.homelab.br
```

Também pode testar pelo IP:

```
ssh -i ~/.ssh/jmsalles-homelab/svc-docker-deploy svc-docker-deploy@192.168.31.37
```

Resultado esperado:

```
Login realizado com sucesso no servidor docker-hml usando chave SSH.
```

Saia do usuário:

```
exit
```

16. Adicionar `svc-docker-deploy` ao grupo `docker`

Neste modelo, o usuário `svc-docker-deploy` terá permissão direta para executar comandos Docker.

Observação importante:

```
Usuário no grupo docker possui privilégio elevado no host.  
Na prática, acesso ao grupo docker pode equivaler a acesso administrativo.  
Para o homelab e esteira controlada, seguiremos com essa abordagem simplificada.
```

Adicione o usuário ao grupo `docker`.

```
sudo usermod -aG docker svc-docker-deploy
```

Valide:

```
id svc-docker-deploy
```

Resultado esperado:

```
svc-docker-deploy deve aparecer como membro do grupo docker.
```

Exemplo:

```
groups=...,docker
```

17. Permitir login SSH com usuário e senha para `svc-docker-deploy`

Neste item, será permitido que o usuário `svc-docker-deploy` acesse o servidor também com senha.

Valide primeiro se o usuário tem senha ativa:

```
sudo passwd -S svc-docker-deploy
```

Se aparecer **L**, o usuário está bloqueado para senha.

Desbloqueie, se necessário:

```
sudo passwd -u svc-docker-deploy
```

Redefina a senha, se necessário:

```
sudo passwd svc-docker-deploy
```

17.1 Criar configuração SSH específica para o usuário

Crie um arquivo dedicado para o usuário:

```
sudo vim /etc/ssh/sshd_config.d/99-svc-docker-deploy.conf
```

Conteúdo:

```
Match User svc-docker-deploy
    PasswordAuthentication yes
    PubkeyAuthentication yes
    KbdInteractiveAuthentication yes
```

Observação:

```
Essa configuração permite senha e chave SSH apenas para o usuário svc-docker-deploy.
```

17.2 Validar configuração do SSH

Valide a configuração:


```
sudo sshd -t
```

Se não retornar erro, recarregue o serviço SSH.

Em Ubuntu/Debian, normalmente:

```
sudo systemctl reload ssh
```

Em Rocky/RHEL, normalmente:

```
sudo systemctl reload sshd
```

Se quiser usar um comando compatível com ambos:

```
sudo systemctl reload ssh || sudo systemctl reload sshd
```

17.3 Testar login com usuário e senha

De outra máquina, teste:

```
ssh svc-docker-deploy@docker-hml.jmsalles.homelab.br
```

Informe a senha criada no item 14.

Também pode testar pelo IP:

```
ssh svc-docker-deploy@192.168.31.37
```

Resultado esperado:

```
Login realizado com sucesso usando usuário e senha.
```

17.4 Testar login com chave novamente

Valide também se a chave continua funcionando:

```
ssh -i ~/.ssh/jmsalles-homelab/svc-docker-deploy svc-docker-deploy@docker-hml.jmsalles.homelab.br
```

Resultado esperado:

```
Login realizado com sucesso usando chave SSH.
```

Parte 5: Testar Docker com **svc-docker-deploy**

18. Testar Docker localmente com o usuário

Entre como o usuário:

```
sudo su - svc-docker-deploy
```

Teste o Docker:

```
docker ps
```

Teste o Docker Compose:

```
docker compose version
```

Teste o compose do Nginx:

```
cd /home/jmsalles/nginx
```

```
docker compose ps
```

Teste o compose do Gitea:

```
cd /home/jmsalles/gitea
```

```
docker compose ps
```

Teste o compose do Nexus:

```
cd /home/jmsalles/nexus
```

```
docker compose ps
```

Teste o compose do Stirling PDF:

```
cd /home/jmsalles/stirling
```

```
docker compose ps
```

Saia do usuário:

```
exit
```

19. Testar Docker via SSH com chave

Na máquina da esteira ou cliente, execute:

```
ssh -i ~/.ssh/jmsalles-homelab/svc-docker-deploy svc-docker-deploy@docker-hml.jmsalles.homelab.br "docker ps"
```

Teste o compose do Gitea:

```
ssh -i ~/.ssh/jmsalles-homelab/svc-docker-deploy svc-docker-deploy@docker-hml.jmsalles.homelab.br "cd /home/jmsalles/gitea && docker compose ps"
```

Teste o compose do Nexus:

```
ssh -i ~/.ssh/jmsalles-homelab/svc-docker-deploy svc-docker-deploy@docker-
```

```
hml.jmsalles.homelab.br "cd /home/jmsalles/nexus && docker compose ps"
```

Teste o compose do Stirling PDF:

```
ssh -i ~/.ssh/jmsalles-homelab/svc-docker-deploy svc-docker-deploy@docker-hml.jmsalles.homelab.br "cd /home/jmsalles/stirling && docker compose ps"
```

20. Testar Docker via SSH com usuário e senha

De uma máquina cliente, execute:

```
ssh svc-docker-deploy@docker-hml.jmsalles.homelab.br "docker ps"
```

Informe a senha.

Teste o compose do Gitea:

```
ssh svc-docker-deploy@docker-hml.jmsalles.homelab.br "cd /home/jmsalles/gitea && docker compose ps"
```

Teste o compose do Nexus:

```
ssh svc-docker-deploy@docker-hml.jmsalles.homelab.br "cd /home/jmsalles/nexus && docker compose ps"
```

Teste o compose do Stirling PDF:

```
ssh svc-docker-deploy@docker-hml.jmsalles.homelab.br "cd /home/jmsalles/stirling && docker compose ps"
```

Parte 6: Como a esteira usará os usuários

21. Uso do **svc-gitea-runner** na pipeline

O **svc-gitea-runner** será usado para login e push no Nexus.

Exemplo conceitual da pipeline:

```
docker login registry.jmsalles.homelab.br -u svc-gitea-runner -p  
"$NEXUS_PASSWORD"
```

```
docker build -t registry.jmsalles.homelab.br/lab/app-teste:latest .
```

```
docker push registry.jmsalles.homelab.br/lab/app-teste:latest
```

A senha do usuário deve ficar futuramente no Vault.

22. Uso do `svc-k8s-pull` no Kubernetes

O `svc-k8s-pull` será usado no `imagePullSecret`.

Exemplo:

```
kubectl create secret docker-registry nexus-registry \  
  --docker-server=registry.jmsalles.homelab.br \  
  --docker-username=svc-k8s-pull \  
  --docker-password='<SENHA_DO_USUARIO>' \  
  --docker-email='jefersonmattossalles@gmail.com' \  
  -n hml
```

No deployment Kubernetes:

```
imagePullSecrets:  
  - name: nexus-registry
```

Imagem:

```
image: registry.jmsalles.homelab.br/lab/app-teste:latest
```

23. Uso do `svc-docker-deploy` na esteira Docker

A esteira poderá acessar o host Docker HML via SSH.

Exemplo usando chave SSH:

```
ssh -i ~/.ssh/jmsalles-homelab/svc-docker-deploy \  
svc-docker-deploy@docker-hml.jmsalles.homelab.br \  
"cd /home/jmsalles/minha-app && docker compose pull && docker compose up -d"
```

Exemplo usando usuário e senha:

```
ssh svc-docker-deploy@docker-hml.jmsalles.homelab.br \  
"cd /home/jmsalles/minha-app && docker compose pull && docker compose up -d"
```

Restart controlado:

```
ssh svc-docker-deploy@docker-hml.jmsalles.homelab.br \  
"cd /home/jmsalles/minha-app && docker compose restart"
```

Ver logs:

```
ssh svc-docker-deploy@docker-hml.jmsalles.homelab.br \  
"cd /home/jmsalles/minha-app && docker compose logs --tail=100"
```

Observação:

Para o Nexus, evite deploy automático sem janela, porque ele é componente central do registry.

Parte 7: Estrutura futura no Vault

Quando o Vault for instalado, guardar os segredos assim:

```
secret/nexus/svc-gitea-runner  
secret/nexus/svc-k8s-pull  
secret/docker/svc-docker-deploy  
secret/gitea/act-runner  
secret/kubernetes/hml
```

Sugestão de conteúdo para `secret/nexus/svc-gitea-runner`:

```
{
  "username": "svc-gitea-runner",
  "password": "<SENHA_DO_USUARIO>",
  "registry": "registry.jmsalles.homelab.br"
}
```

Sugestão de conteúdo para `secret/nexus/svc-k8s-pull`:

```
{
  "username": "svc-k8s-pull",
  "password": "<SENHA_DO_USUARIO>",
  "registry": "registry.jmsalles.homelab.br"
}
```

Sugestão de conteúdo para `secret/docker/svc-docker-deploy`:

```
{
  "username": "svc-docker-deploy",
  "host": "docker-hml.jmsalles.homelab.br",
  "auth_type": "ssh-key-and-password"
}
```

Parte 8: Checklist final

Item	Status
Role <code>role-docker-hosted-push</code> criada no Nexus	Pendente
Role <code>role-docker-hosted-pull</code> criada no Nexus	Pendente
Usuário <code>svc-gitea-runner</code> criado no Nexus	Pendente
Usuário <code>svc-k8s-pull</code> criado no Nexus	Pendente
Docker Bearer Token Realm habilitado	Pendente
Login com <code>svc-gitea-runner</code> testado	Pendente
Push com <code>svc-gitea-runner</code> testado	Pendente
Login com <code>svc-k8s-pull</code> testado	Pendente
Pull com <code>svc-k8s-pull</code> testado	Pendente
Push negado para <code>svc-k8s-pull</code> validado	Pendente
Usuário Linux <code>svc-docker-deploy</code> criado	Pendente

Item	Status
Senha do <code>svc-docker-deploy</code> definida	Pendente
Chave SSH privada e pública criadas	Pendente
Chave pública adicionada ao <code>authorized_keys</code>	Pendente
Login SSH por chave testado	Pendente
Login SSH por usuário e senha testado	Pendente
Usuário <code>svc-docker-deploy</code> adicionado ao grupo <code>docker</code>	Pendente
Comando <code>docker ps</code> testado com <code>svc-docker-deploy</code>	Pendente
<code>docker compose ps</code> testado nos diretórios do ambiente	Pendente

24. Resumo final

Usuários criados no Nexus:

```
svc-gitea-runner  
svc-k8s-pull
```

Usuário criado no Linux do Docker HML:

```
svc-docker-deploy
```

Permissões do `svc-docker-deploy`:

```
Login via chave SSH  
Login via usuário e senha  
Membro do grupo docker  
Permissão direta para docker e docker compose
```

Fluxo de publicação:

```
Gitea act_runner  
|  
svc-gitea-runner  
|  
registry.jmsalles.homelab.br  
|  
docker-hosted no Nexus
```


Fluxo de consumo Kubernetes:

```
Kubernetes
|
svc-k8s-pull
|
registry.jmsalles.homelab.br
|
docker-hosted no Nexus
```

Fluxo de deploy Docker:

```
Pipeline
|
SSH com svc-docker-deploy
|
docker-hml.jmsalles.homelab.br
|
Grupo docker
|
docker compose pull
docker compose up -d
docker compose restart
docker compose logs
```

Criado por Jeferson Salles LinkedIn: <https://www.linkedin.com/in/jmsalles/> E-mail: jefersonmattossalles@gmail.com GitHub: <https://github.com/jmsalles/>